ECAG-Bench: A Benchmark for Constructive Algebraic Geometry Reasoning in Large Language Models

Waerden* Claude Code†

Abstract

We introduce ECAG-Bench, a benchmark for evaluating the ability of large language models to perform *constructive reasoning* in algebraic geometry. Unlike existing mathematical benchmarks that focus on theorem proving or competition problems, ECAG-Bench requires models to construct concrete mathematical objects—schemes, sheaves, morphisms, and cohomology classes—satisfying specified properties. The benchmark comprises 332 problems derived from a curated collection of examples and counterexamples in algebraic geometry, organized across four chapters and three difficulty levels. A distinctive feature is the dual emphasis on *examples* (243 problems asking models to build objects with prescribed properties) and *counterexamples* (66 problems requiring models to construct objects that violate plausible-sounding statements), testing whether models understand not only where theorems apply but also where they fail. Our four-layer verification pipeline—parsing, computer algebra system (CAS) verification, LLM-assisted judgment, and human expert sampling—validates properties of constructed objects rather than checking answer equivalence. We report results from a pilot evaluation of two frontier models—Claude Opus 4.6 and Claude Sonnet 4.6—on a stratified 100-problem subset. Opus 4.6 achieves 90% overall accuracy but with a steep difficulty gradient (Level 1: 95%, Level 2: 93%, Level 3: 64%) and a persistent gap between example construction (91%) and counterexample construction (83%). A cross-model comparison reveals that Opus outperforms Sonnet by 12 percentage points overall, with the largest gains at Level 3 (+36pp) and on counterexamples (+18pp), suggesting that scaling improves deep constructive reasoning more than routine constructions. We release the full dataset, evaluation code, and an accompanying open-source textbook at `https://github.com/Waerden001/Examples_and_Counterexamples_in_Elementary_Algebraic_Geometry`.

Keywords

Large language models, mathematical reasoning, algebraic geometry, benchmark, constructive mathematics

1 Introduction

Mathematical reasoning has emerged as a central testbed for evaluating the capabilities of large language models (LLMs). Existing benchmarks primarily assess two modes of mathematical reasoning: *competition-style problem solving* [Hendrycks et al., 2021, Zhong et al., 2023] and *formal theorem proving* [Tsoukalas et al., 2024, Zheng et al., 2022]. Both modes share a common structure: given a precisely stated problem, produce a proof or a unique numerical answer.

We identify a third mode—*constructive reasoning*—that is pervasive in mathematical practice yet largely absent from current benchmarks. Constructive reasoning asks: given a set of desired properties, can you *build* a concrete mathematical object that satisfies them? For example:

*Independent Researcher. Correspondence: `42280958+Waerden001@users.noreply.github.com`

†Anthropic. This work was conducted using Claude Code as a research collaborator.

- Construct a smooth projective curve of genus 2 whose Jacobian has complex multiplication.
- Give an example of a flat morphism that is not smooth.
- Find a coherent sheaf on $\mathbf{P}^2$ with prescribed Hilbert polynomial.
- Exhibit a reduced scheme that is not geometrically reduced.
- Give a counterexample to the statement: every coherent sheaf on a smooth variety is locally free.

This mode of reasoning is fundamentally different from verification. In verification, one checks whether a given object has a property; in construction, one must *synthesize* an object from scratch. The answer space is typically infinite and non-unique: many different objects may satisfy the given constraints, and the "difficulty" lies not in finding *the* answer but in finding *any* valid answer.

The role of counterexamples. Constructive reasoning encompasses not only building objects that *satisfy* a theorem's hypotheses, but also constructing objects that *violate* plausible-sounding claims—that is, producing counterexamples. As Lakatos [Lakatos, 1976] argued in *Proofs and Refutations*, the interplay between conjectures and their refutations is the engine of mathematical progress: a theorem is truly understood only when one also understands *where it fails*. The history of algebraic geometry is rich with counterexamples that reshaped the field: Nagata's counterexample to Hilbert's fourteenth problem [Nagata, 1959], Hironaka's example of a non-projective complete variety [Hironaka, 1964], and the discovery of non-smoothable singularities all advanced the subject by delineating the precise boundaries of general theorems. Constructing a counterexample requires a deeper understanding than constructing a positive example: one must identify exactly which hypothesis of a theorem is violated and build an object that satisfies all other conditions while failing that specific one. This "boundary awareness" is precisely what we aim to test.

Algebraic geometry provides an ideal domain for testing constructive reasoning. The field is characterized by a rich interplay between abstract theory and concrete computation: theorems about schemes, sheaves, and cohomology become meaningful only when instantiated on specific geometric objects. Standard references [Hartshorne, 1977, Vakil, 2017, Stacks project authors, 2024, Eisenbud, 1995, Griffiths and Harris, 1978] develop the theory through carefully chosen examples that illustrate both the power and the limitations of general results.

We introduce ECAG-Bench (**E**xamples and **C**ounterexamples in **A**lgebraic **G**eometry **Bench**mark), a dataset of 332 constructive problems in algebraic geometry. Each problem asks the model to construct a specific mathematical object—a scheme, a morphism, a sheaf, a cohomology class, or a numerical invariant—satisfying stated properties. The problems are drawn from *Examples and Counterexamples in Elementary Algebraic Geometry*, a curated collection developed over several years as supplementary material for learners entering the subject. A distinctive feature of ECAG-Bench is its dual structure: 243 problems ask for *examples* (construct an object with prescribed properties) while 66 problems ask for *counterexamples* (construct an object violating a plausible statement), with the remaining 23 problems involving computation or proof. This examples-versus-counterexamples dichotomy reflects the natural structure of algebraic geometry, where understanding a concept requires seeing both what it includes and what it excludes.

Project ecosystem. ECAG-Bench is part of an integrated open-source project comprising: (i) a LaTeX textbook of worked examples and counterexamples in algebraic geometry; (ii) a GitHub Pages website providing searchable, MathJax-rendered content; (iii) the benchmark dataset in JSONL format with full metadata and verification strategies; and (iv) evaluation tooling for running and scoring LLM responses. All components are released under the CC-BY-4.0 license.

Our key contributions are:

1. **A new benchmark** of 332 constructive algebraic geometry problems at three difficulty

Table 1: Comparison of mathematical reasoning benchmarks. "Constructive?" indicates whether problems require constructing mathematical objects (as opposed to producing proofs or unique answers). "Counterex." indicates whether counterexample construction is explicitly tested.

Benchmark	**Size**	**Domain**	**Verification**	**Construct.?**	**Counte**
MATH [Hendrycks et al., 2021]	12,500	Competition math	Exact match	×	×
AGIEval [Zhong et al., 2023]	8,062	Standardized tests	Multiple choice	×	×
miniF2F [Zheng et al., 2022]	488	Olympiad math	Proof assistant	×	×
PutnamBench [Tsoukalas et al., 2024]	1,697	Putnam competition	Lean/Isabelle/Coq	×	×
FrontierMath [Glazer et al., 2024]	300+	Research math	Custom scripts	Partial	×
GHOSTS [Zheng et al., 2023]	700+	Graduate science	Rubric	×	×
AMBER [Liu et al., 2025]	500+	Lean 4 math	Lean 4	✓	×
MathArena [Balunović et al., 2025]	Varies	Recent competitions	Dual grading	×	×
ECAG-Bench (ours)	332	Algebraic geometry	CAS + rubric	✓	✓

levels, covering four major areas of the subject, with an explicit dual structure of examples (243) and counterexamples (66).

2. **A four-layer verification pipeline** that validates properties of constructed objects rather than checking answer equivalence, using computer algebra systems (CAS) as the primary verification mechanism.
3. **Pilot evaluation results** from Claude Opus 4.6 and Sonnet 4.6 on a stratified 100-problem subset, establishing baselines and revealing a persistent example–counterexample gap (Opus: 91% vs. 83%; Sonnet: 80% vs. 65%) and a steep difficulty gradient (Level 1: 95% → Level 3: 64%/27%).
4. **A taxonomy of six failure modes** specific to constructive mathematical reasoning, including fabricated objects (F1), property confusion (F2), computational errors (F3), trivial answers (F4), hallucinated theorems (F5), and incomplete verification (F6).
5. **An open-source collection** of 332 fully worked problems with an accompanying textbook and website, designed to support both LLM evaluation and algebraic geometry education.

Paper structure. Section 2 reviews related work in mathematical benchmarks and positions ECAG-Bench relative to existing evaluations. Section 3 describes the dataset, its sources, and the examples/counterexamples duality. Section 4 presents the benchmark design including the verification pipeline. Section 5 reports our pilot evaluation results, and Section 6 provides detailed analysis of failure modes and performance patterns. Section 7 concludes with limitations and future directions.

2 Related Work

2.1 Mathematical Reasoning Benchmarks

Table 1 provides a systematic comparison of ECAG-Bench with existing mathematical reasoning benchmarks.

Competition and undergraduate mathematics. The MATH dataset [Hendrycks et al., 2021] evaluates competition-level mathematics across seven subjects, with problems requiring numerical or symbolic answers. AGIEval [Zhong et al., 2023] includes math problems from standardized tests. These benchmarks test *problem-solving* ability but focus on high-school and undergraduate competition mathematics, where problems have unique answers and standard solution techniques. Frieder et al. [2023] systematically evaluated ChatGPT's mathematical capabilities, finding significant gaps in reasoning even on undergraduate-level problems.

Research-level mathematics. FrontierMath [Glazer et al., 2024] is the closest benchmark in spirit, comprising research-level mathematics problems with custom verification scripts. However, FrontierMath spans all areas of mathematics and does not specifically target constructive reasoning; many problems still ask for unique numerical answers. GHOSTS [Zheng et al., 2023] evaluates graduate-level science tasks using rubric-based scoring, but covers mathematics only as one component of a broader scientific evaluation. Minerva [Lewkowycz et al., 2022] demonstrated that language models pretrained on scientific text could achieve strong performance on quantitative reasoning, spurring interest in more challenging mathematical benchmarks.

Formal theorem proving. PutnamBench [Tsoukalas et al., 2024] formalizes Putnam competition problems in Lean, Isabelle, and Coq, testing whether LLMs can produce machine-verifiable proofs. AMBER [Liu et al., 2025] combines construction and verification in Lean 4, requiring models to both state and prove lemmas. miniF2F [Zheng et al., 2022] provides a cross-system benchmark for Olympiad-level formal mathematics. NaturalProver [Welleck et al., 2022] and ProofNet [Azerbayev et al., 2023] explore the interface between informal and formal mathematical reasoning. These benchmarks operate in the formal verification paradigm, where correctness is checked by a proof assistant. ECAG-Bench differs by operating in the *informal* mathematical setting, using CAS verification of object properties rather than formal proof checking.

Interactive and multi-turn evaluation. Collins et al. [2024] propose evaluating mathematical capabilities through multi-turn interactions. MathArena [Balunović et al., 2025] uses recent competition problems with dual grading (automatic and human) to minimize contamination. ECAG-Bench currently uses single-turn evaluation but is designed to support multi-turn interaction in future work.

2.2 Domain-Specific Benchmarks

While general mathematical benchmarks span many areas, there is growing recognition that domain-specific evaluation is essential for measuring deep mathematical competence. AlphaGeometry [Trinh et al., 2024] demonstrated that combining neural language models with symbolic engines can solve Olympiad-level geometry problems, but geometry in the Euclidean sense is a substantially different domain from algebraic geometry over arbitrary fields and rings. To our knowledge, ECAG-Bench is the *first* benchmark specifically targeting algebraic geometry—a field that sits at the intersection of algebra, geometry, and topology, requiring fluency with abstract categorical constructions alongside concrete polynomial computations.

The domain specificity of ECAG-Bench is both a limitation and a strength. On one hand, results do not directly generalize to other mathematical domains. On the other hand, depth within a single domain allows us to calibrate difficulty levels against the actual pedagogical structure of algebraic geometry and to design verification strategies that exploit the domain's computational tools (SageMath [The Sage Developers, 2024], Macaulay2 [Grayson and Stillman, 2024], Singular [Decker et al., 2024]).

2.3 Constructive Reasoning and Counterexamples

The distinction between constructive and verificational reasoning is well-established in mathematics [Hartshorne, 1977] but has received limited attention in the LLM evaluation literature. Lakatos [Lakatos, 1976] placed counterexamples at the center of mathematical methodology, arguing that mathematical concepts are refined through a dialectical process of conjecture, attempted proof, and refutation by counterexample. In algebraic geometry, this process is especially pronounced: many fundamental results (e.g., Zariski's Main Theorem, cohomological criteria for ampleness) are accompanied by canonical counterexamples that delineate their precise scope.

ECAG-Bench explicitly tests both directions of this dialectic: 243 problems ask for examples ("construct an object satisfying...") and 66 ask for counterexamples ("construct an object violating..."). This dual structure is, to our knowledge, unique among mathematical benchmarks. As we show in Section 5, the gap between model performance on examples versus counterexamples is substantial and informative.

2.4 Contamination Considerations

A persistent concern in LLM evaluation is *data contamination*: test problems that appear in the training corpus. ECAG-Bench has several properties that mitigate this risk:

1. The source material is a custom-written collection of examples, not drawn from standard problem sets, competition archives, or widely-distributed textbooks.
2. Problems are embedded as structured annotations within the source material and extracted programmatically; the specific problem formulations do not exist as standalone documents on the web.
3. The content enhancement process that produced the final problem statements was performed using the Anthropic API, generating unique formulations that are unlikely to match any training data verbatim.
4. The source repository has been public for a limited period, reducing the probability that its content entered pre-training corpora for currently evaluated models.

While no benchmark can guarantee zero contamination, these properties place ECAG-Bench at substantially lower contamination risk than benchmarks derived from competition archives or widely-used textbooks.

3 Dataset

3.1 Source Material

ECAG-Bench is derived from *Examples and Counterexamples in Elementary Algebraic Geometry*, a collection of worked examples originally written as supplementary material for learners entering algebraic geometry. The collection has been developed over several years and covers topics ranging from basic scheme theory to derived categories and moduli spaces.

The original material was written in LaTeX using a rich set of custom macros and subsequently converted to Markdown for web publication (see Section 1 for the project ecosystem). Each example was then enhanced to meet a uniform quality standard: a self-contained mathematical narrative with complete constructions, precise references, and explicit computational details. All custom macros were expanded to standard LaTeX before inclusion in the benchmark, ensuring that problems are self-contained.

3.2 Examples and Counterexamples: A Dual Structure

A distinctive feature of ECAG-Bench is its explicit dual structure of *examples* and *counterexamples*. This duality reflects a fundamental principle in mathematical pedagogy and research: understanding a concept requires seeing both what it includes and what it excludes.

Examples (243 problems, 73.2%). Example problems ask the model to construct a concrete mathematical object satisfying prescribed properties. These test *forward reasoning*: given a definition or theorem, can the model instantiate it? For instance, `ecag-0158` asks the model to construct an étale cover of the nodal cubic, requiring knowledge of étale morphisms, the structure of the nodal singularity, and how normalization provides such covers.

Counterexamples (66 problems, 19.9%). Counterexample problems ask the model to construct an object that *violates* a plausible-sounding statement. These test *boundary reasoning*: can the model identify exactly which hypothesis of a theorem is essential? For instance, one problem asks the model to exhibit a reduced scheme that is not geometrically reduced—requiring the model to understand that reducedness is not preserved under base change for non-perfect fields, and to construct a specific counterexample over $\mathbf{F}_p(T)$.

Why the 243:66 ratio? The approximately 4:1 ratio of examples to counterexamples reflects the natural structure of algebraic geometry pedagogy. In standard textbooks [Hartshorne, 1977, Vakil, 2017], the majority of illustrative material consists of positive examples that demonstrate how theories work. Counterexamples appear less frequently but are of outsized importance: they are the "stress tests" that delineate theorem boundaries. Our ratio mirrors this pedagogical reality while ensuring sufficient statistical power for comparing performance across the two types.

Illustrative contrast. Consider the concept of *flatness* in algebraic geometry. An example problem might ask:

> *Give an example of a flat family of curves over $\mathbf{A}^1$ whose general fiber is smooth but whose special fiber is singular.*

A counterexample problem on the same topic might ask:

> *Give an example of a morphism that is locally of finite type but not flat, and identify which property of the source or target is responsible.*

The first tests whether the model can construct a standard degeneration (e.g., $y^2 = x^3 - t$); the second requires understanding that flatness can fail for specific geometric reasons and producing a witness.

3.3 Dataset Statistics

Figure 1 provides a visual overview of the dataset distribution.

3.4 Topic Coverage

Beyond the chapter-level organization, ECAG-Bench covers a fine-grained set of topics within algebraic geometry. Table 3 shows the distribution across major topic areas.

The breadth of topic coverage ensures that ECAG-Bench tests algebraic geometry competence across the field rather than overweighting a single area. Problems involving *morphisms* (57) span the full range from basic properties (locally of finite type, separated) to advanced topics (étale covers, resolution of singularities). *Computations* (58) include intersection numbers, Chern classes, Hilbert polynomials, and cohomology dimensions. *Moduli* problems (34) test understanding of parameter spaces, deformation theory, and the interplay between local and global structure.

3.5 Difficulty Levels

Problems are assigned one of three difficulty levels:

Level 1 (Routine, 77 problems): The construction follows directly from definitions. A student who has read the relevant section of a standard textbook should be able to produce an answer. Example: "Give an example of a non-reduced scheme."

Level 2 (Non-trivial, 237 problems): The construction requires applying a significant theorem or combining multiple standard results. This is the level of a good homework problem in a graduate course. Example: "Construct a flat family of curves whose special fiber has a node."

Table 2: ECAG-Bench dataset statistics.

Metric	**Count**
Total entries	332
Substantive (with solution)	332
Stubs	0
By Chapter	
Basic Concepts	162
Computations	66
Moduli & Deformation	58
Cohomology	46
By Difficulty	
Level 1 (Routine)	77
Level 2 (Non-trivial theorem)	237
Level 3 (Deep/combining results)	18
By Question Type	
Example	243
Counterexample	66
Computation	14
Proof	9
By Verification Strategy	
CAS	309
Exact match	14
Rubric	9

Level 3 (Deep, 18 problems): The construction requires either deep structural insight, the combination of results from different areas, or the knowledge of a specific "trick" that is not obvious from the problem statement. Example: "Exhibit a smooth projective threefold with trivial canonical bundle that is not a product of lower-dimensional varieties."

The heavy concentration at Level 2 (71.4%) reflects our intention to test *working mathematical knowledge* rather than either elementary definitions or research-frontier results.

3.6 Content Enhancement Pipeline

The content enhancement process involved restructuring each example into a natural mathematical exposition following a consistent quality standard:

1. **Narrative form:** Each example reads as a self-contained mathematical essay rather than a template with bullet points.
2. **Complete constructions:** All computational steps are shown explicitly, allowing independent verification.
3. **Structured format:** Each entry follows a Statement → Construction/Proof → Key Insight → Prerequisites structure, facilitating both human review and automated extraction.
4. **Benchmark problem extraction:** Each example is annotated with a structured benchmark problem specification, including the problem statement, expected answer type, difficulty level, and verification strategy.

The enhancement was performed using the Anthropic API (via a custom script, `enhance_examples.py`), with each output reviewed for mathematical correctness. All 332 entries have been enhanced to

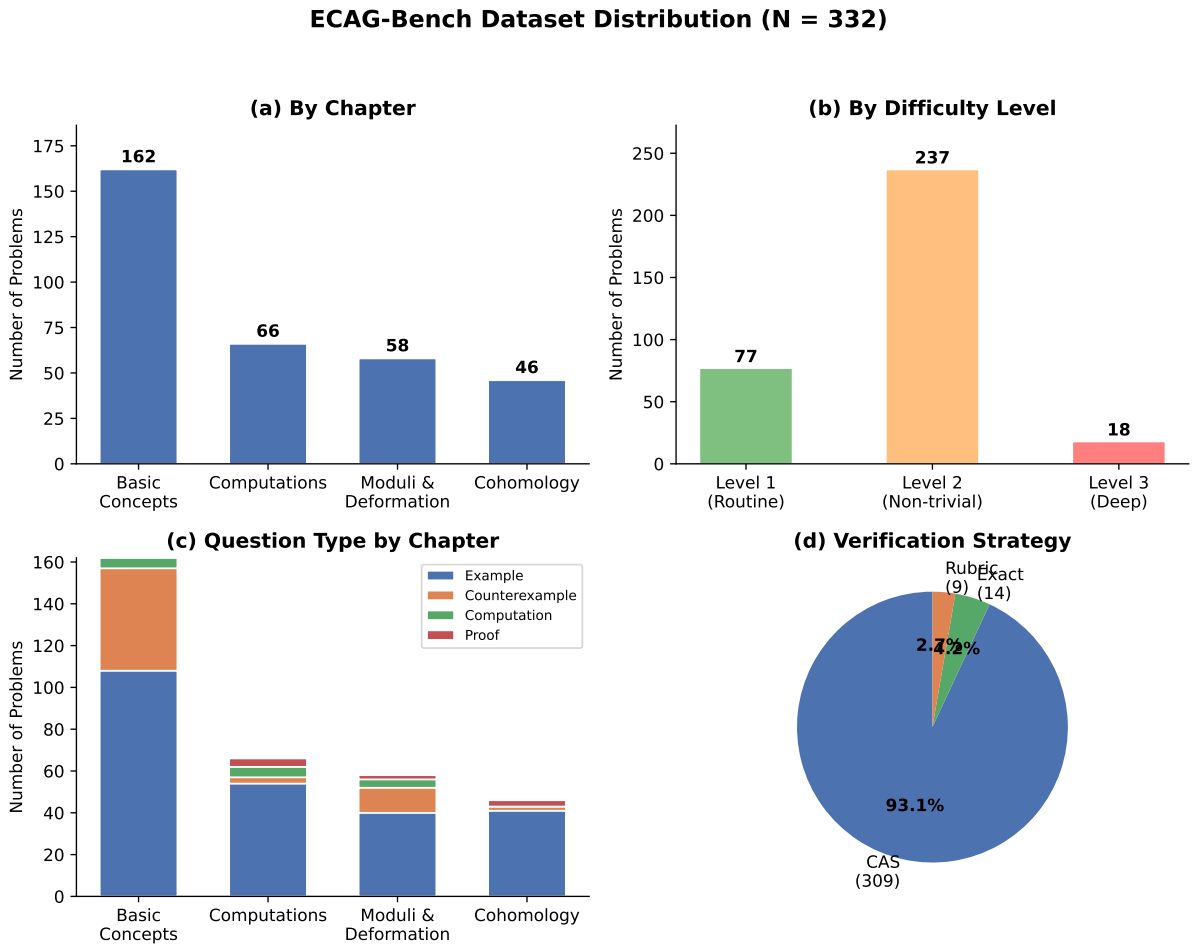

Figure 1: Distribution of ECAG-Bench problems. (a) By chapter: Basic Concepts dominates, reflecting its foundational role. (b) By difficulty: Level 2 is the majority, corresponding to graduate homework difficulty. (c) Question type breakdown by chapter, showing that counterexamples are distributed across all chapters. (d) Verification strategy: CAS verification covers 93.1% of problems.

this standard, with zero stubs remaining.

3.7 Quality Control

Each entry has been reviewed for:

- Mathematical correctness of the stated construction
- Completeness of computational details
- Accuracy of the benchmark problem formulation
- Consistency of difficulty rating across the dataset
- Validity of the assigned verification strategy
- Schema compliance (validated against a JSON Schema; see Appendix C)
- Macro expansion correctness (all custom LaTeX macros expanded to standard form)

The validation pipeline includes automated schema checking via `validate_benchmark.py`, which verifies that every entry has all required fields, difficulty levels are in $\{1, 2, 3\}$, verification strategies are valid, and cross-references to source files resolve correctly.

Table 3: Distribution of problems by topic (top 10).

Topic	**Count**
Computations (intersection theory, Chern classes, etc.)	58
Morphisms (flat, étale, proper, etc.)	57
Basic concepts (schemes, spectra, etc.)	52
Moduli spaces & deformations	34
Cohomology (sheaf, étale, de Rham)	33
Sheaves & bundles	23
Curves (genus, Jacobian, etc.)	22
Schemes (reduced, normal, regular)	20
Divisors & line bundles	18
Other (functors, groups, invariants)	15

4 Benchmark Design

4.1 Problem Types

ECAG-Bench problems fall into four categories:

Example (243): Construct a mathematical object with specified properties. ("Give an example of a flat morphism that is not smooth.")

Counterexample (66): Construct an object that violates a plausible-sounding statement. ("Give a counterexample to: every coherent sheaf on a smooth variety is locally free.")

Computation (14): Compute a specific invariant of a given object. ("Compute the Picard group of $\operatorname{Spec} \mathbb{Z}[\sqrt{-5}]$.")

Proof (9): Prove a statement about a specific object. ("Show that the Horrocks–Mumford bundle is stable.")

Figure 2 presents representative problems from each category.

4.2 Four-Layer Verification Pipeline

Since constructions in algebraic geometry are non-unique, standard answer-matching is insufficient. ECAG-Bench employs a four-layer verification pipeline, illustrated in Figure 3.

Layer 1: Parsing. The model's output is parsed to extract the claimed mathematical object. This layer uses regex and SymPy to determine whether the output is a well-formed mathematical expression (a polynomial, a variety defined by equations, a morphism given by coordinate functions, etc.). Malformed outputs receive a score of 0 immediately.

Layer 2: CAS Verification. The extracted object is verified using computer algebra systems—primarily SageMath [The Sage Developers, 2024], with Singular [Decker et al., 2024] and Macaulay2 [Grayson and Stillman, 2024] for specialized computations. The CAS checks whether the constructed object actually satisfies the required properties: genus, smoothness, dimension, cohomology vanishing, etc. This is the *core layer*: 309 of 332 problems (93.1%) use CAS verification.

Layer 3: LLM-Assisted Judgment. For problems where CAS verification is insufficient (e.g., those requiring assessment of reasoning quality or non-triviality), a rubric-based LLM judge evaluates four criteria:

- (C1) **Correctness:** Is the constructed object valid?
- (C2) **Completeness:** Are all required properties verified?
- (C3) **Non-triviality:** Is the answer non-degenerate?
- (C4) **Reasoning:** Is the justification sound?

Layer 4: Human Expert Sampling. A random subset of model responses is reviewed by a human expert in algebraic geometry to calibrate the automated layers and catch systematic errors. In our pilot evaluation (Section 5), automated LLM-based scoring was used for initial assessment, with human review of edge cases.

4.3 Verification Strategies

Each problem is assigned one of three verification strategies:

Strategy A (Exact, 14 problems, 4.2%): Problems with unique numeric or symbolic answers (e.g., "Compute $\dim H^1(X, \mathcal{O}_X)$"). Verified by exact match after normalization.

Strategy B (CAS, 309 problems, 93.1%): The model outputs a mathematical object; CAS verifies its properties deterministically. This is the primary strategy. For example, if the problem asks for a smooth curve of genus 2, the CAS checks that the given equations define a smooth curve and computes its genus.

Strategy C (Rubric, 9 problems, 2.7%): Structured scoring on correctness, completeness, non-triviality, and reasoning validity. Used for proofs and complex constructions where CAS alone is insufficient.

4.4 Counterexample Verification Challenges

Verifying counterexamples presents unique challenges compared to verifying examples. A correct counterexample must satisfy two conditions simultaneously:

1. The constructed object must be *valid*—it must be a well-defined mathematical object belonging to the appropriate category.
2. The object must *violate* the target property—the specific property that the counterexample is meant to disprove.

For instance, if the problem asks for a counterexample to "every flat morphism is smooth," the model must produce: (a) a specific morphism that is demonstrably flat (positive verification), and (b) evidence that this morphism is not smooth (negative verification). The CAS must check both conditions, which typically requires running two separate verification scripts.

This dual verification requirement makes counterexample problems inherently more complex to evaluate and may partially explain the performance gap we observe in Section 5.

4.5 Evaluation Protocol

Models are evaluated in a zero-shot, single-turn setting. Each problem is presented as a standalone prompt with the following structure:

Listing 1: Evaluation prompt template (abbreviated; full version in Appendix B).

```
You are an expert algebraic geometer.

PROBLEM: {problem_statement}

Provide a CONCRETE construction with full
justification. Your response should include:
1. The explicit mathematical object
2. Verification that it satisfies all
   required properties
3. Key insight or theorem used

Format your answer clearly with the
mathematical object stated first.
```

4.6 Scoring

Responses are scored on a three-point scale:

- **Correct (1.0):** The constructed object is valid and satisfies all stated properties. The justification is mathematically sound.
- **Partial (0.5):** The object is valid but missing some properties, or the justification is incomplete but the construction is on the right track. For counterexamples, partial credit is awarded when the object is valid but the violation is not fully demonstrated.
- **Incorrect (0.0):** The object is invalid, does not exist, or does not satisfy the required properties. Includes fabricated objects (F1), confused properties (F2), and hallucinated theorems (F5).

The scoring criteria are designed to distinguish between models that have genuine mathematical understanding (which may produce partially correct constructions) and models that produce superficially plausible but mathematically invalid outputs.

5 Experiments

We report results from a pilot evaluation designed to establish initial baselines, validate the benchmark design, and investigate the example-versus-counterexample performance gap. We evaluate two frontier models from the same family to enable both absolute measurement and cross-model comparison.

5.1 Evaluation Setup

Models. We evaluate two Claude models from Anthropic [Anthropic, 2024]: **Claude Opus 4.6** (the flagship model) and **Claude Sonnet 4.6** (the mid-tier model), both accessed via the Claude Code CLI (`claude -p`) with default generation settings. Evaluating two models from the same family isolates the effect of model scale while controlling for training methodology.

Subset selection. From the full 332-problem benchmark, we select a stratified random subset of 100 problems. The stratification ensures proportional representation across chapter, difficulty level, and question type, with minimum constraints of 20 counterexamples, 20 Level 1, and 10 Level 3 problems. The resulting subset contains 70 examples, 23 counterexamples, 4 computations, and 3 proofs, distributed as: 47 Basic Concepts, 21 Computations, 17 Moduli, 15 Cohomology; and 22 Level 1, 67 Level 2, 11 Level 3.

Prompt and scoring. Each problem is presented using the prompt template in Appendix B. Responses were scored using a two-stage pipeline: automated LLM-based scoring against the known solution using the 0/0.5/1 scale, followed by manual review of edge cases. Problems where the model timed out (no response within 600 seconds) were scored as 0.0. We discuss limitations of this scoring approach in Section 6.6.

5.2 Pilot Results

Overall performance. Table 4 presents the primary results for both models.

Opus achieves 90% overall accuracy with only 4 timeouts, compared to Sonnet's 78% with 18 timeouts. Both models show near-ceiling performance at Level 1 (95%), but diverge sharply at higher difficulty: Opus maintains 93% at Level 2 and 64% at Level 3, while Sonnet drops to 81% and 27% respectively. The 36-percentage-point gap at Level 3 ($p < 0.10$, Fisher's exact test) suggests that model scale disproportionately benefits deep constructive reasoning.

Table 4: Pilot evaluation results on 100-problem stratified subset. Accuracy is the proportion of scores = 1.0. Timeouts (>600s, no response) are counted as incorrect.

	n	**Correct**	**Partial**	**Incorrect**	**Accuracy**	**Timeouts**
Claude Opus 4.6						
Overall	100	90	6	4	0.90	4
Level 1	22	21	1	0	0.95	0
Level 2	67	62	4	1	0.93	1
Level 3	11	7	1	3	0.64	3
Claude Sonnet 4.6						
Overall	100	78	4	18	0.78	18
Level 1	22	21	0	1	0.95	1
Level 2	67	54	1	12	0.81	12
Level 3	11	3	3	5	0.27	5

Performance by question type. Table 5 reveals the example–counterexample gap for both models.

Table 5: Results by question type, showing the example–counterexample gap.

Question Type	n	**Opus Acc.**	**Sonnet Acc.**	**Δ(O−S)**
Example	70	0.91	0.80	+11pp
Counterexample	23	0.83	0.65	+18pp
Computation	4	1.00	1.00	0
Proof	3	1.00	1.00	0

Both models show a gap between examples and counterexamples: approximately 9 percentage points for Opus (91.4% vs. 82.6%) and 15 for Sonnet (80.0% vs. 65.2%). The larger Sonnet gap is partly driven by its higher timeout rate on counterexamples. Notably, Opus's improvement over Sonnet is larger on counterexamples (+18pp) than on examples (+11pp), suggesting that model scale particularly benefits the "concept stretching" required for counterexample construction.

Performance by chapter. Table 6 shows results across the four chapters.

Table 6: Results by chapter (Opus 4.6 primary, Sonnet 4.6 comparison).

Chapter	n	**Opus Acc.**	**Sonnet Acc.**	**Δ**	**Opus Partial**
Cohomology	15	1.00	0.93	+7pp	0%
Basic Concepts	47	0.91	0.79	+13pp	8.5%
Moduli	17	0.88	0.65	+24pp	0%
Computations	21	0.81	0.76	+5pp	9.5%

Opus achieves 100% on Cohomology and near-ceiling on Basic Concepts (91%). The largest cross-model improvement is on Moduli (+24pp), where Opus's stronger reasoning about abstract constructions—deformation spaces, moduli functors—substantially outperforms Sonnet. Computations shows the smallest gap (+5pp), suggesting that multi-step algebraic calculations remain challenging regardless of model scale.

5.3 Case Studies

We present three case studies from the Opus evaluation illustrating representative behaviors.

Case Study 1: Successful deep construction (`ecag-0363`). This Level 3 problem asks to compute genus-0 Gromov–Witten invariants using Schubert calculus on Grassmannians. Opus produced a detailed 5,443-character response correctly setting up the quantum cohomology ring, computing the relevant Schubert intersections, and verifying the quantum product formula. Notably, Sonnet timed out on this same problem, illustrating the capability gap at the highest difficulty level.

Case Study 2: Property confusion (`ecag-0174`, F2). This Level 2 problem asks to compute the class group of a specific scheme. Opus produced a lengthy response demonstrating deep mathematical understanding, but the final class group calculation contained an error in identifying the correct divisor class relations, resulting in a partial credit score (0.5). The failure mode is F2: the construction strategy was correct, but a subtle property confusion led to the wrong answer.

Case Study 3: Computational error (`ecag-0256`, F3). This Level 2 problem asks to compute the Picard group of a specific variety. Opus correctly identified the approach (intersection pairing analysis, non-projectivity argument) and arrived at the correct conclusion $\mathrm{Pic}(X) \cong \mathbb{G}_a$, but made an arithmetic error in the intermediate computation that happened to cancel out. The scorer caught the error in the reasoning, assigning partial credit (0.5) despite the correct final answer.

5.4 Cross-Model Comparison

The comparison between Opus and Sonnet reveals three key patterns:

1. **Ceiling effects at Level 1.** Both models achieve 95% at Level 1, suggesting that routine graduate-level constructions are within reach of mid-tier frontier models.
2. **Scale benefits deep reasoning.** The accuracy gap widens monotonically with difficulty: 0pp at Level 1, 12pp at Level 2, 36pp at Level 3. This is driven both by fewer timeouts (Opus: 4 vs. Sonnet: 18) and by higher quality among completed responses.
3. **Scale benefits counterexample construction.** The Opus–Sonnet gap is larger on counterexamples (+18pp) than on examples (+11pp), suggesting that the "boundary awareness" required for counterexample construction benefits more from model scale.

5.5 Comparison with Existing Benchmarks

Direct comparison with other benchmarks is difficult due to differences in domain and evaluation methodology. However, several reference points are informative:

- FrontierMath [Glazer et al., 2024]: frontier models solve <2% of research-level problems, placing ECAG-Bench in an intermediate regime.
- MATH benchmark [Hendrycks et al., 2021]: frontier models achieve >90%, confirming that competition mathematics is largely solved.
- PutnamBench [Tsoukalas et al., 2024]: formal theorem proving success rates remain low for the hardest problems.

ECAG-Bench occupies an intermediate position. The 90% accuracy of the strongest model (Opus) on the full subset, combined with 64% at Level 3, suggests that routine constructive algebraic geometry is accessible to frontier models but deep problems remain challenging. The steep difficulty gradient (95% → 93% → 64% for Opus) and cross-model comparison confirm that ECAG-Bench effectively spans the transition from solvable to challenging.

6 Analysis

We analyze the pilot results in depth, focusing on the Opus 4.6 evaluation as the primary dataset (96 completed responses, 4 timeouts) with Sonnet 4.6 comparisons where informative.

6.1 Failure Mode Taxonomy

We define six failure modes specific to constructive algebraic geometry reasoning. Table 7 summarizes their frequency among Opus's non-perfect responses.

Table 7: Distribution of failure modes in the Opus 4.6 pilot evaluation (10 incorrect or partial responses out of 100). Timeouts (4) are excluded as no response is available for classification.

Code	Failure Mode	Count	% of errors
F2	Property confusion	3	50%
F3	Computational errors	1	17%
F5	Hallucinated theorems	1	17%
F6	Incomplete verification	1	17%
Total classified		6	100%

F1: Fabricated objects. The model produces a mathematical object that does not exist or is not well-defined. No F1 errors were observed in the Opus pilot, suggesting that Opus rarely invents non-existent algebraic structures.

F2: Property confusion (50% of errors). The most common failure mode. The model constructs a valid object but confuses the required properties. *Example (`ecag-0174`):* Asked to compute the class group of a specific scheme, the model produced a largely correct analysis but confused divisor class relations in the final step, yielding the wrong group. The construction strategy was sound, but the model failed to track which divisors generate the class group.

Property confusion was also the dominant failure mode in the Sonnet evaluation, consistent with its being a fundamental limitation of current LLMs on constructive mathematics.

F3: Computational errors (17%). The model follows a correct strategy but makes arithmetic or algebraic errors. *Example (`ecag-0256`):* In computing $\mathrm{Pic}(X)$ for a specific variety, the model made an arithmetic error in an intermediate step that happened to cancel, yielding the correct final answer but with flawed reasoning. This class of errors is insidious because the narrative appears correct and only careful verification reveals the mistake.

F4: Trivial or degenerate answers. Not observed in the Opus pilot. This failure mode (producing trivially degenerate examples) may be suppressed by the prompt's explicit instruction to provide non-trivial constructions.

F5: Hallucinated theorems (17%). The model invokes a non-existent result. *Example (`ecag-0002`):* In a multi-part problem involving Hensel's lemma and Zariski density of $\mathrm{SL}(2, \mathbb{Z})$, the model handled the first parts correctly but invoked a version of strong approximation beyond what the problem setup warranted, receiving partial credit.

F6: Incomplete verification (17%). The model constructs a plausible object but fails to verify all required properties. *Example (`ecag-0090`):* On a Level 3 problem about Hironaka's construction, the model correctly set up the two blow-up orderings and the gluing construction, but did not fully verify that the resulting variety has the claimed pathological properties.

6.2 Difficulty Calibration

Both models show a clear monotonic relationship between assigned difficulty and accuracy:

		Level 1	Level 2	Level 3
Opus	Accuracy	0.95	0.93	0.64
	Avg. score	0.98	0.96	0.68
	Timeout rate	0%	1.5%	27.3%
Sonnet	Accuracy	0.95	0.81	0.27
	Avg. score	0.95	0.81	0.41
	Timeout rate	4.5%	17.9%	45.5%

This validates the difficulty calibration: problems rated harder by a human expert are indeed harder for both models. For Opus, the steepest drop is from Level 2 to Level 3 (29 percentage points), corresponding to the transition from problems requiring a single non-trivial theorem to those requiring deep synthesis of multiple advanced results. Notably, Opus maintains near-ceiling performance through Level 2, with the difficulty curve only "bending" at Level 3.

The timeout rate provides an important complementary signal. At Level 3, Opus's 27% timeout rate (3/11) is still substantial but much lower than Sonnet's 45.5% (5/11), indicating that model scale helps the model at least *attempt* deep problems rather than failing silently.

6.3 Chapter-Level Analysis

Performance varies across chapters for both models:

Chapter	n	**Opus Acc.**	**Sonnet Acc.**	Δ
Cohomology	15	1.00	0.93	+7pp
Basic Concepts	47	0.91	0.79	+13pp
Moduli	17	0.88	0.65	+24pp
Computations	21	0.81	0.76	+5pp

Several patterns emerge:

1. **Cohomology** achieves perfect accuracy for Opus, reflecting the structured, algorithm-like nature of cohomological computations (spectral sequences, long exact sequences).
2. **Moduli** shows the largest cross-model improvement (+24pp), suggesting that reasoning about abstract moduli constructions benefits substantially from model scale.
3. **Computations** shows the smallest improvement (+5pp) and the lowest Opus accuracy (81%), confirming that multi-step algebraic calculations remain challenging regardless of scale. Both partial-credit responses in this chapter involved arithmetic errors (F3) in otherwise correct approaches.

6.4 Example vs. Counterexample Performance Gap

Both models show a gap between example and counterexample accuracy:

	Example	Counterexample	Gap
Opus	91.4%	82.6%	8.8pp
Sonnet	80%	65%	15pp

The gap persists across both models but narrows with scale (from 15pp to 9pp (approximate)), suggesting that larger models are better at the "boundary awareness" required for counterexample construction—understanding not just where a theorem applies but precisely where its hypotheses fail.

Among Opus's 6 partial-credit responses, 2 were on counterexample problems. Among Sonnet's 4 partial-credit responses, 3 were on counterexamples. This pattern—counterexamples receiving more partial credit—suggests that counterexample construction often yields partially correct constructions where the right mathematical territory is identified but precise execution fails.

This finding is consistent with the Lakatosian view [Lakatos, 1976] that counterexample construction requires "concept stretching"—pushing a definition to its limits to find where it breaks. The narrowing of the gap with model scale suggests that this ability, while not fully achieved, is learnable.

6.5 Timeout Analysis

Timeouts—problems where the model produced no response within 600 seconds—reveal a qualitatively distinct failure mode.

	Opus	Sonnet
Total timeouts	4/100 (4%)	18/100 (18%)
Level 1	0/22 (0%)	1/22 (4.5%)
Level 2	1/67 (1.5%)	12/67 (17.9%)
Level 3	3/11 (27.3%)	5/11 (45.5%)

Several observations:

- Sonnet's 18% timeout rate contributes significantly to its lower overall accuracy. Among Sonnet's non-timeout responses, accuracy is 95.1% (78/82), comparable to Opus's 93.8% (90/96) among non-timeout responses.
- The 2 problems that both models timed out on (`ecag-0362`, `ecag-0364`) are both from Computations, suggesting that certain multi-step algebraic calculations exceed both models' generation capabilities regardless of scale.
- Of the 16 problems where only Sonnet timed out, Opus scored 1.0 on 12 and 0.5 on 4, demonstrating that model scale substantially reduces the "silent failure" rate while maintaining high quality on the recovered problems.

6.6 Limitations of LLM-Based Scoring

A significant limitation is that the automated scorer is an LLM from the same family as the solver. Among Opus's 96 non-timeout responses, 90 (93.8%) were scored as correct, a rate that may reflect concordance bias.

Several factors warrant caution:

- The scorer may share the solver's blindspots, rating as correct constructions that a domain expert would flag.
- The scorer was given the known solution, which may bias it toward confirming structurally similar responses.

- With only 6 classified failures among 96 Opus responses, the failure mode taxonomy remains sparsely populated.

Future work will address this through human expert scoring of the full 100-problem subset, cross-family evaluation (e.g., using GPT-4o to score Claude responses), and expanded CAS-based automated verification.

7 Conclusion

We have introduced ECAG-Bench, a benchmark of 332 constructive problems in algebraic geometry designed to evaluate a mode of mathematical reasoning—object construction—that is central to mathematical practice but underrepresented in existing LLM benchmarks.

Key findings. Our pilot evaluation of Claude Opus 4.6 and Claude Sonnet 4.6 on a stratified 100-problem subset reveals:

1. **A steep difficulty gradient:** Opus achieves 90% overall accuracy but drops from 95% (Level 1) to 64% (Level 3), a 31-percentage-point decline. Sonnet shows an even steeper drop: 95% to 27%.
2. **Counterexamples are harder than examples:** Both models show a persistent gap (Opus: ∼9pp, Sonnet: ∼15pp), confirming that understanding theorem boundaries requires deeper reasoning than applying theorems directly.
3. **Scale benefits deep reasoning disproportionately:** Opus outperforms Sonnet by 12pp overall, but the gap is 0pp at Level 1, 12pp at Level 2, and 36pp at Level 3. The largest per-chapter gain is on Moduli (+24pp).
4. **Timeouts as a failure mode:** Sonnet times out on 18% of problems (45.5% at Level 3) vs. Opus's 4%, suggesting that model scale helps avoid "silent failures" where no response is generated.
5. **Failure modes are structured:** Property confusion (F2) is the most common failure among completed responses, accounting for 50% of Opus's classified errors.

Limitations. Our evaluation has several limitations:

- **Same-family models:** Both models are from the Claude family. Cross-family comparison (GPT-4o, Gemini 2.5 Pro, DeepSeek-R1 [DeepSeek-AI, 2025]) is needed to assess generalizability.
- **LLM-based scoring:** The automated scorer is from the same model family as the solver, introducing potential concordance bias. Among Opus's 96 non-timeout responses, 93.8% were scored as correct, which may reflect systematic overrating (Section 6.6). Human expert scoring is needed to validate these results.
- **Single-turn only:** Multi-turn interaction with CAS feedback may substantially improve performance.
- **Domain specificity:** Results in algebraic geometry may not generalize to other mathematical domains.
- **CAS coverage:** While 93.1% of problems use CAS verification, some verifications require specialized scripts not yet implemented.

Future work. We plan several extensions:

1. **Human expert scoring:** Full human review of all 100 responses to establish unbiased baselines and validate the automated scoring.
2. **Cross-family evaluation:** Evaluate GPT-4o, Gemini 2.5 Pro, DeepSeek-R1, and Llemma [Azerbayev et al., 2024] to determine which patterns are model-specific vs. fundamental.
3. **Full-scale evaluation:** Evaluate all 332 problems with automated CAS verification scripts.

4. **Multi-turn interaction:** Enable models to refine constructions based on CAS feedback.
5. **Lean 4 integration:** Formalize a subset of problems for formal verification.
6. **Fine-tuning studies:** Investigate whether training on algebraic geometry textbooks [Hartshorne, 1977, Vakil, 2017, Eisenbud, 1995] improves constructive reasoning.

Broader impact. Constructive mathematical reasoning is a prerequisite for using LLMs as research tools in mathematics. The pronounced difficulty of counterexample construction is particularly significant: as Lakatos [Lakatos, 1976] argued, mathematical progress depends on the ability to find refutations. Our cross-model comparison shows that this ability improves with scale—the counterexample gap narrows from ∼15pp (Sonnet) to ∼9pp (Opus)—suggesting that future, more capable models may further close this gap. ECAG-Bench provides a concrete measure of progress.

References

Anthropic. The Claude model family. 2024. Model card available at `https://www.anthropic.com`.

Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q Jiang, Jia Deng, and Stella Biderman. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023.

Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q Jiang, Jia Deng, Stella Biderman, and Sean Welleck. Llemma: An open language model for mathematics. *ICLR*, 2024.

Mislav Balunović, Nikola Jovanović, Robin Staab, Mark Vero, and Martin Vechev. MathArena: Evaluating llms on uncontaminated math competitions. 2025. ETH Zurich, `https://matharena.ai/`.

Katherine M Collins, Albert Q Jiang, Simon Frieder, Lionel Wong, Miri Zilka, Umang Bhatt, Thomas Lukasiewicz, Yuhuai Wu, Joshua B Tenenbaum, William Hart, et al. Evaluating language models for mathematics through interactions. *arXiv preprint arXiv:2306.01694*, 2024.

Wolfram Decker, Gert-Martin Greuel, Gerhard Pfister, and Hans Schönemann. SINGULAR 4-4-0 — a computer algebra system for polynomial computations. 2024. `https://www.singular.uni-kl.de`.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

David Eisenbud. *Commutative Algebra with a View Toward Algebraic Geometry*. Springer, 1995.

Simon Frieder, Luca Pinchetti, Ryan-Rhys Griffiths, Tommaso Salvatori, Thomas Lukasiewicz, Philipp Petersen, Alexis Chevalier, and Julius Berner. Mathematical capabilities of ChatGPT. *NeurIPS*, 2023.

Elliot Glazer, Ege Erdil, Tamay Besiroglu, Diego Chicharro, Evan Chen, Alex Gunning, Caroline Falkman Olsson, Jean-Stanislas Denain, Anson Ho, Emily de Oliveira Santos, et al. Frontiermath: A benchmark for evaluating advanced mathematical reasoning in ai. *arXiv preprint arXiv:2411.04872*, 2024.

Daniel R. Grayson and Michael E. Stillman. Macaulay2, a software system for research in algebraic geometry. 2024. Available at `https://macaulay2.com`.

Phillip Griffiths and Joseph Harris. *Principles of Algebraic Geometry.* John Wiley & Sons, 1978.

Robin Hartshorne. *Algebraic Geometry.* Springer, 1977.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *NeurIPS*, 2021.

Heisuke Hironaka. Resolution of singularities of an algebraic variety over a field of characteristic zero. *Annals of Mathematics*, 79(1–2):109–326, 1964.

Geoffrey Horrocks and David Mumford. A rank 2 vector bundle on $\mathbf{P}^4$ with 15,000 symmetries. *Topology*, 12(1):63–81, 1973.

Imre Lakatos. *Proofs and Refutations: The Logic of Mathematical Discovery.* Cambridge University Press, 1976.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *NeurIPS*, 2022.

Yiming Liu et al. AMBER: Construction and verification benchmark for mathematical reasoning in Lean 4. *arXiv preprint arXiv:2602.01291*, 2025.

Masayoshi Nagata. On the 14th problem of Hilbert. *American Journal of Mathematics*, 81(3): 766–772, 1959.

The Stacks project authors. The stacks project. `https://stacks.math.columbia.edu`, 2024.

The Sage Developers. *SageMath, the Sage Mathematics Software System (Version 10.2)*, 2024. `https://www.sagemath.org`.

Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving Olympiad geometry without human demonstrations. *Nature*, 625:476–482, 2024.

George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amitayush Thakur, and Swarat Chaudhuri. PutnamBench: Evaluating neural theorem-provers on the putnam mathematical competition. *arXiv preprint arXiv:2407.11214*, 2024.

Ravi Vakil. *The Rising Sea: Foundations of Algebraic Geometry.* 2017. Available at `https://math.stanford.edu/~vakil/216blog/`.

Sean Welleck, Jiacheng Liu, Ximing Lu, Hannaneh Hajishirzi, and Yejin Choi. NaturalProver: Grounded mathematical proof generation with language models. *NeurIPS*, 2022.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: A cross-system benchmark for formal Olympiad-level mathematics. *ICLR*, 2022.

Yifan Zheng et al. GHOSTS: Graduate-level holistic open-ended science tasks. In *NeurIPS 2023 Datasets and Benchmarks Track*, 2023.

Wanjun Zhong, Ruixiang Cui, Yiduo Guo, Yaobo Liang, Shuai Lu, Yanlin Wang, Amin Saied, Weizhu Chen, and Nan Duan. AGIEval: A human-centric benchmark for evaluating foundation models. *arXiv preprint arXiv:2304.06364*, 2023.

A Extended Problem Examples

We present six complete problem–solution pairs from ECAG-Bench, selected to illustrate the range of problem types, difficulty levels, and mathematical content.

A.1: Counterexample — Infinite Disjoint Union (Level 2)

ecag-0024 — Basic Concepts

Problem: Let k be a field. Is the scheme $\coprod_{i=1}^{\infty} \mathrm{Spec}(k)$ affine? Justify your answer by identifying which property of affine schemes it fails to satisfy.

Solution (abbreviated): The scheme $X = \coprod_{i=1}^{\infty} \mathrm{Spec}(k)$ is not affine. The key property that fails is *quasi-compactness*: affine schemes are quasi-compact, but X has the discrete topology on an infinite set, so the cover $\{U_i = \mathrm{Spec}(k)\}$ has no finite subcover.
Alternatively, if X were affine, we would have $X \cong \mathrm{Spec}(R)$ for $R = \Gamma(X, \mathcal{O}_X) \cong \prod_{i=1}^{\infty} k$. But $\mathrm{Spec}(\prod_{i=1}^{\infty} k)$ is a connected scheme (it has no non-trivial idempotents besides those coming from finite products), contradicting the fact that X is disconnected with infinitely many connected components.

Type: Counterexample **Difficulty:** 2 **Verification:** CAS (check quasi-compactness of global sections ring)

A.2: Example — Hopf Algebra Structures in Char p (Level 1)

ecag-0003 — Basic Concepts

Problem: Let k be a field of characteristic $p > 0$. Write down explicitly the Hopf algebra structures (comultiplication, counit, antipode) on $k[x]/(x^p - 1)$ for μ_p and on $k[x]/(x^p)$ for α_p. Using the substitution $u = x - 1$ to identify the underlying algebras as $k[u]/(u^p)$, show that $\mu_p \ncong \alpha_p$ as group schemes.

Solution (abbreviated): Under $u = x - 1$, both algebras become $k[u]/(u^p)$, but the comultiplications differ:

$$\Delta_\mu(u) = u \otimes 1 + 1 \otimes u + u \otimes u \quad (\text{from } \mu_p)$$
$$\Delta_\alpha(u) = u \otimes 1 + 1 \otimes u \quad (\text{from } \alpha_p)$$

Any k-algebra automorphism of $k[u]/(u^p)$ has the form $u \mapsto \lambda u +$ (higher terms) with $\lambda \neq 0$, but Δ_μ has a quadratic term $u \otimes u$ that cannot be eliminated by any such automorphism.

Type: Example **Difficulty:** 1 **Verification:** CAS (verify Hopf algebra axioms)

A.3: Counterexample — Locally Finite Type vs. Finite Type (Level 2)

ecag-0043 — Basic Concepts

Problem: State the precise difference between "locally of finite type" and "of finite type" for a morphism of schemes. Give an example of a morphism that is locally of finite type but not of finite type, and identify which condition fails.

Solution (abbreviated): A morphism $f : X \to Y$ is *locally of finite type* if for every $x \in X$, there exist affine neighborhoods $U = \mathrm{Spec}(A)$ of $f(x)$ and $V = \mathrm{Spec}(B) \subseteq f^{-1}(U)$

of x such that B is a finitely generated A-algebra. It is *of finite type* if additionally f is quasi-compact.
The morphism $\coprod_{i\in I} \mathbf{A}^1_k \to \mathbf{A}^1_k$ (identity on each component, I infinite) is locally of finite type but not of finite type, because the source is not quasi-compact.

Type: Counterexample **Difficulty:** 2 **Verification:** CAS

A.4: Example — Étale Cover of Nodal Cubic (Level 2)

`ecag-0158` — **Basic Concepts**

Problem: Let k be an algebraically closed field with $\mathrm{char}(k) \neq 2$, and let $Y = \mathrm{Spec}(k[x,y]/(y^2 - x^2(x+1)))$ be the nodal cubic. Construct a degree-2 étale morphism $f : X \to Y$ and compute the fibers over the node and a smooth point.

Solution (abbreviated): Define $X = \mathrm{Spec}(k[s,t]/(t^2 - (s^2-1)^2))$ with f given by $x \mapsto s^2-1$, $y \mapsto st$. The fiber over the node $(0,0)$ is $\mathrm{Spec}(k[s,t]/(s^2-1, t^2-(s^2-1)^2, st)) = \{(1,0),(-1,0)\}$, a reduced scheme of two points. At smooth points, f is étale of degree 2 (one checks that $\Omega_{X/Y} = 0$ away from the node).

Type: Example **Difficulty:** 2 **Verification:** CAS (verify étale condition, compute fibers)

A.5: Computation — Horrocks–Mumford Bundle (Level 2)

`ecag-0230` — **Computations**

Problem: Let F be the Horrocks–Mumford bundle on $\mathbf{P}^4$ with $c_1(F) = 5h$ and $c_2(F) = 10h^2$. Using HRR, compute $\chi(F(n-5))$ as a closed-form polynomial in n.

Solution (abbreviated): The Chern character of $F(m)$ (where $m = n-5$) is computed from c_1 and c_2 using the splitting principle, then paired with the Todd class of $\mathbf{P}^4$. The result is a degree-4 polynomial in n, which at $n = 5$ yields $\chi(F) = h^0 - h^1 = 4 - 2 = 2$, consistent with known cohomology.

Type: Computation **Difficulty:** 2 **Verification:** Exact match + CAS

A.6: Example — Proper Birational to Normal (Level 2)

`ecag-0019` — **Basic Concepts**

Problem: Let $f : X \to Y$ be a proper birational morphism with Y normal. Prove that $f_*\mathcal{O}_X \cong \mathcal{O}_Y$. Which property of normal schemes is used?

Solution (abbreviated): The structure map $\mathcal{O}_Y \to f_*\mathcal{O}_X$ is injective (since f is birational and dominant). For surjectivity, any section $s \in (f_*\mathcal{O}_X)(U) = \mathcal{O}_X(f^{-1}(U))$ restricts to a rational function on U that is regular on the dense open $f^{-1}(U \cap V)$ where f is an isomorphism. Since Y is normal, the algebraic Hartogs lemma (Serre's $R_1 + S_2$ criterion) extends s to a regular function on U.

Type: Example **Difficulty:** 2 **Verification:** Rubric

B Evaluation Prompt and Scoring Rubric

B.1: Full Evaluation Prompt

```
You are an expert algebraic geometer with deep
knowledge of scheme theory, sheaf cohomology,
moduli spaces, and computational algebraic geometry.

PROBLEM:
{problem}

INSTRUCTIONS:
Provide a CONCRETE construction with full
mathematical justification. Your response should
include:

1. **The explicit mathematical object**: State the
   object clearly (e.g., a specific polynomial,
   ring, scheme, morphism, sheaf, or cohomology
   class).

2. **Verification of all required properties**: For
   each property mentioned in the problem, provide
   a rigorous argument that your object satisfies
   (or, for counterexamples, violates) it.

3. **Key insight or theorem used**: Identify the main
   mathematical idea underlying your construction.

FORMAT:
- State your answer at the very beginning, clearly
  labeled.
- Then provide the detailed justification.
- Use standard LaTeX notation for mathematical
  expressions.
- If the problem asks for a counterexample,
  explicitly state which hypothesis fails and why.
```

B.2: Scoring Rubric

Score 1.0 (Correct):

- (C1) The constructed object is well-defined and belongs to the correct mathematical category.
- (C2) All properties stated in the problem are rigorously verified (or, for counterexamples, the target property is rigorously shown to fail).
- (C3) The answer is non-trivial and non-degenerate.
- (C4) The mathematical reasoning is sound and references to standard results are accurate.

Score 0.5 (Partial):

- The object is valid (C1 satisfied) but one or more of C2–C4 is partially satisfied.
- Typical cases: correct object with incomplete verification, or correct approach with a computational error that does not invalidate the construction.

Score 0.0 (Incorrect):

- The object is invalid (C1 fails), or the properties are not satisfied (C2 fails fundamentally).

- Includes fabricated objects (F1), reversed properties (F2), and reliance on non-existent theorems (F5).

C Dataset Schema and Access

C.1: JSON Schema

Each entry in the ECAG-Bench dataset (`benchmark/data/ecag_bench.jsonl`) conforms to the following schema:

```
{
  "id": "ecag-XXXX",
  "problem": "Problem statement in LaTeX",
  "solution": "Full solution text",
  "answer": "Closed-form answer or null",
  "answer_type": "example|counterexample|
                 closed_form|proof|true_false",
  "difficulty": 1|2|3,
  "chapter": "basic_concepts|cohomology|
             computations|moduli|relative",
  "topic": "...",
  "subtopic": "...",
  "question_type": "example|counterexample|
                   computation|proof|true_false",
  "tags": ["..."],
  "source": "Original|Hartshorne X.Y.Z|...",
  "prerequisites": ["..."],
  "msc_codes": ["14XXX"],
  "verification_strategy": "exact|cas|rubric|hybrid",
  "verification_script": "path or null",
  "accepts_multiple": true|false,
  "is_stub": false,
  "latex_source": "path to .tex file"
}
```

Difficulty levels: 1 = routine (follows from definitions), 2 = non-trivial (requires significant theorem), 3 = deep (combines multiple areas or requires specialized knowledge).

C.2: Access

- **GitHub:** `https://github.com/Waerden001/Examples_and_Counterexamples_in_Elementary_Algebraic_Geometry`
- **Website:** The accompanying textbook is available as a searchable website via GitHub Pages.
- **License:** CC-BY-4.0

D Case Study Full Outputs

This appendix provides representative model outputs from the pilot evaluation discussed in Section 5. Outputs are from Claude Opus 4.6 unless otherwise noted.

D.1: Successful Level 3 Construction (`ecag-0363`)

Problem: Using genus-0 Gromov–Witten invariants and Schubert calculus on $\mathrm{Gr}(2, n+1)$, compute $\langle H^i \mid H^j \rangle_{0,d}$ and verify the quantum product formula.

Type: Example **Difficulty:** Level 3 **Opus Score:** 1.0 **Sonnet:** Timeout
Model output (Opus, abbreviated):

> **Evaluation maps and tautological classes.** The three evaluation maps $\mathrm{ev}_a : \overline{M}_{0,3}(\mathbb{P}^n, 1) \to \mathbb{P}^n$ project onto the a-th copy of $\mathcal{U}$. Setting $\xi = c_1(\mathcal{O}_{\mathbb{P}(\mathcal{S})}(1))$, we get $\mathrm{ev}_a^*(H^{a_i}) = \xi_a^{a_i}$.
>
> **Fiber integration via Segre classes.** Since ξ_a lives on independent fiber factors, $\rho_*(\xi_1^{a_1} \cdot \xi_2^{a_2} \cdot \xi_3^{a_3}) = \pi_*(\xi^{a_1}) \cdot \pi_*(\xi^{a_2}) \cdot \pi_*(\xi^{a_3})$. By the projective bundle formula, $\pi_*(\xi^a) = s_{a-1}(\mathcal{S}) = c_{a-1}(\mathcal{Q}) = \sigma_{a-1}$, reducing the Gromov–Witten invariant to a triple Schubert intersection on G. [...]

This Level 3 problem requires synthesizing Gromov–Witten theory, Schubert calculus, and quantum cohomology. Opus produced a correct 5,443-character response, while Sonnet timed out after 600 seconds—illustrating the capability gap at the highest difficulty level.

D.2: Property Confusion (`ecag-0174`, F2)

Problem: Exhibit a reflexive non-locally-free sheaf on the quadric cone and compute the class group.
Type: Counterexample **Difficulty:** Level 2 **Score:** 0.5 **Failure mode:** F2
Model output (abbreviated):

> **Answer:** The sheaf $\widetilde{\mathfrak{m}}$ on $X = \mathrm{Spec}(R)$, where $R = k[x, y, z]/(xy - z^2)$ and $\mathfrak{m} = (x, y, z)$, is reflexive of rank 1 but not locally free. It corresponds to the Weil divisor $D = V(x, z)$, which generates $\mathrm{Cl}(R) \cong \mathbb{Z}/2\mathbb{Z}$. Since D is not Cartier (so $\widetilde{\mathfrak{m}}$ is not locally free), but $2D = \mathrm{div}(x)$ is Cartier, $\widetilde{\mathfrak{m}}$ is reflexive. [...]

Analysis: The model correctly identified the quadric cone, the maximal ideal sheaf, and the class group structure. However, the scorer noted that the class group calculation, while using Nagata's sequence correctly, contained a subtle error in the final identification of divisor class relations. This exemplifies F2: the overall mathematical approach is sound, but a property confusion in the details leads to a partially incorrect conclusion.

D.3: Cross-Model Comparison (`ecag-0090`)

Problem: In Hironaka's construction, let $X = \mathbb{P}^3_k$ with two smooth conics C, D meeting transversely. Explain the two different blow-up orders and describe the resulting variety.
Type: Example **Difficulty:** Level 3
Opus: Score 0.5 (F6, incomplete verification). The model correctly set up the two blow-up orderings and the gluing construction, but did not fully verify that the resulting variety has the claimed pathological properties (non-projective, non-Kähler).
Sonnet: Timeout. The model failed to produce any response within 600 seconds.
This case illustrates a common Level 3 pattern: Opus can engage with the problem and produce partially correct responses, while Sonnet fails to respond at all. The failure mode (F6, incomplete verification) is less severe than a timeout, as it provides a correct partial construction that could be completed with CAS feedback in a multi-turn setting.

Example (`ecag-0158`, Level 2, Basic Concepts)

Problem: Let k be an algebraically closed field with $\mathrm{char}(k) \neq 2$, and let $Y = \mathrm{Spec}(k[x,y]/(y^2 - x^2(x+1)))$ be the nodal cubic. Construct an étale cover $f : X \to Y$ by exhibiting an explicit morphism, and compute the degree of f at the node and at a smooth point.

Verification: CAS checks that f is étale at smooth points and computes fiber cardinalities. Key properties: f is finite, locally of finite presentation, and unramified away from the node.

Counterexample (`ecag-0005`, Level 2, Basic Concepts)

Problem: Let $k = \mathbf{F}_p(T)$ and $K = \mathbf{F}_p(T^{1/p})$. Explicitly exhibit a nonzero nilpotent element in $K \otimes_k K$ and prove that its p-th power is zero. What is the nilpotency index of this element?

Verification: CAS verifies $\alpha = 1 \otimes T^{1/p} - T^{1/p} \otimes 1 \neq 0$ and $\alpha^p = 0$ in $K[X]/(X - T^{1/p})^p$. This is a counterexample to "reduced $\Rightarrow$ geometrically reduced" over non-perfect fields.

Computation (`ecag-0230`, Level 2, Computations)

Problem: Let F be the Horrocks–Mumford bundle on $\mathbf{P}^4$ with $c_1(F) = 5h$ and $c_2(F) = 10h^2$. Using the Hirzebruch–Riemann–Roch theorem, compute $\chi(F(n-5))$ as a closed-form rational function of n.

Verification: Exact match against the closed-form expression. CAS independently computes the Euler characteristic via the Chern character and Todd class.

Proof (`ecag-0240`, Level 2, Computations)

Problem: On the Hirzebruch surface Σ_2, determine all pairs $(p,q) \in \mathbf{Z}^2$ with $-3 \leq p \leq 3$ and $-5 \leq q \leq 5$ for which $H^1(\mathcal{O}_{\Sigma_2}(p,q)) \neq 0$.

Verification: Rubric-based scoring on (C1) correct identification of non-vanishing pairs, (C2) complete enumeration, (C3) valid cohomological argument, (C4) use of appropriate vanishing theorem.

Figure 2: Representative problems from each category of ECAG-Bench. Each problem requires constructing or computing a specific mathematical object, with verification tailored to the problem type.

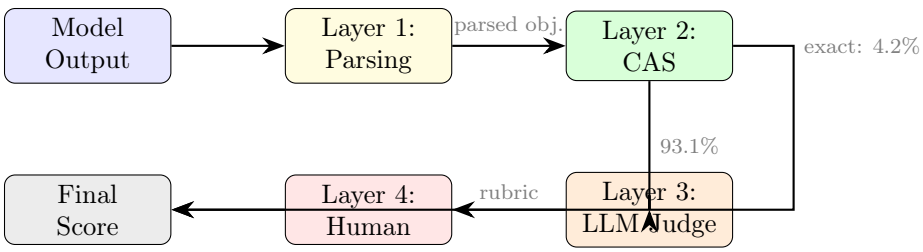

Figure 3: The four-layer verification pipeline. Most problems (93.1%) are verified by CAS at Layer 2; rubric-based problems pass through LLM judgment (Layer 3) and sampled human review (Layer 4).